

Chapter 5. Persisting Data





Outline

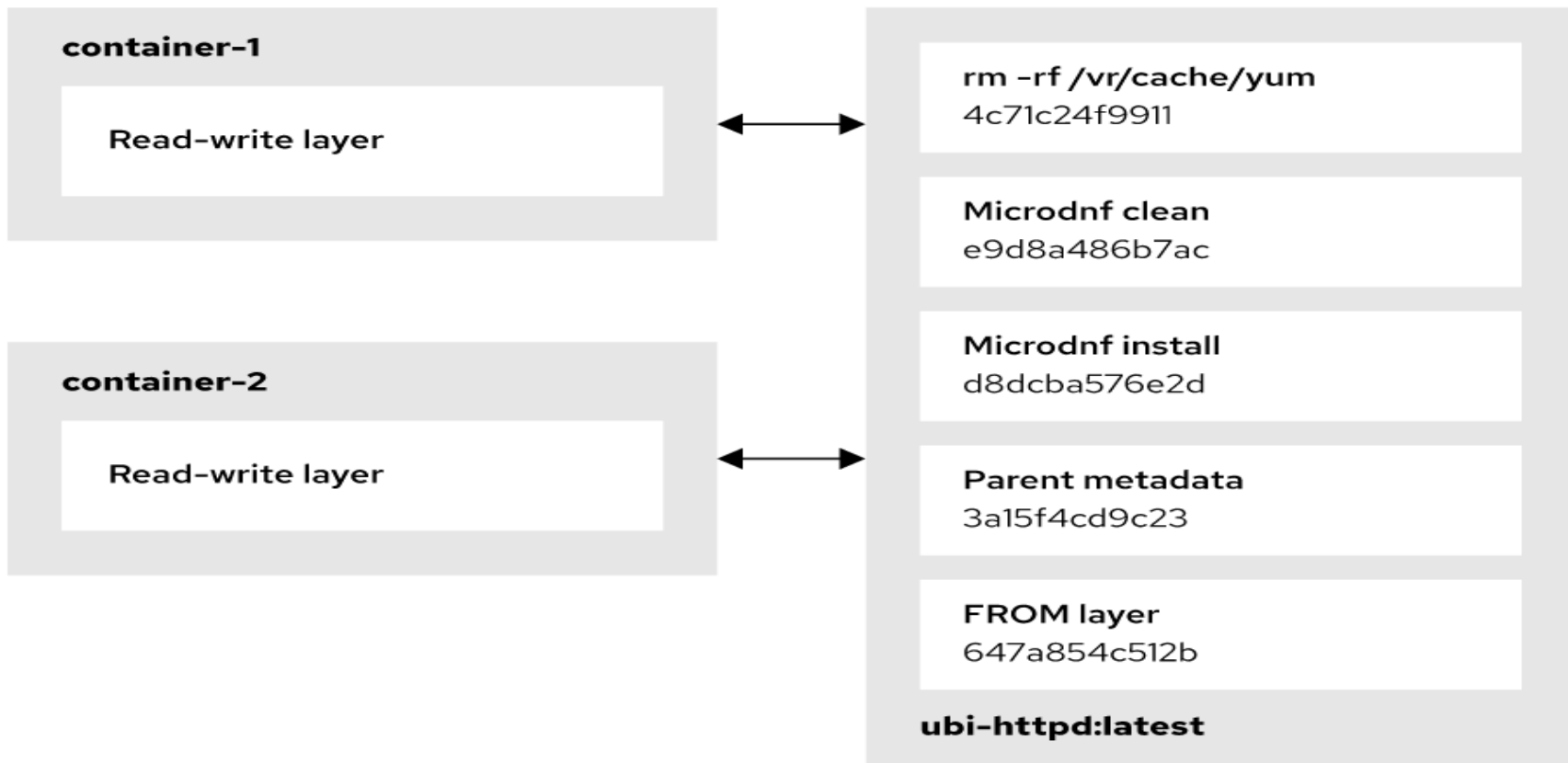
- Describe the process for mounting volumes and common use cases.
- Build containerized databases.



Copy-on-write File System

- When you **build** a container **image**, **each instruction** that modifies the container file system creates a **new read-only data layer**. Because you cannot modify data in an existing layer, **each layer contains a set of changes**, or diffs, **from the previous layer**.
- When you instantiate the **container image**, for example by using the **podman run** command, Podman **creates a thin read-write container layer** on top of the previous layers.
- This means that **multiple containers** that use **one container image** share **read-only layers** and **differ in the runtime read-write layer**. When you **delete a container**, the **read-write layer is destroyed**. This means container runtime data is **ephemeral**.

Multiple containers share immutable read-only layers





Implications of a COW File System

- Modifying a file or its attributes at runtime is possible in several steps:
 - 1) Podman **locates the requested file** in the closest (highest) layer and **copies** it to the **runtime layer**
 - 2) In the **runtime layer**, when the file is available for both **read and write operations**, a containerized process **can modify the file**.
- The implementation of how files behave when a process attempts to read or write a file depends on the **storage driver**.
- Docker uses **storage drivers** to **store image layers, and to store data in the writable layer of a container**. The container's writable layer does not persist after the container is deleted, but is suitable for storing ephemeral data that is generated at runtime.
- Storage drivers are optimized for **space efficiency**, but (depending on the storage driver) write speeds are lower than native file system performance, especially for storage drivers that use a copy-on-write filesystem. (Ex: overlay)



Store Data on Host Machine

- Podman implements external mounts by using **volumes** and **bind mounts**.
- **This is useful for the following reasons:**
 - 1) **Persistence:** Mounted data is persistent across container deletions. Because containers are ephemeral, data from the runtime read-write layer is not accessible after container deletion.
 - 2) **Use of Host File System:** Mounted data typically does not implement the COW file system. Consequently, write-heavy containerized processes can write data to a mount without the limitations of the COW file systems for write operations.
 - 3) **Ease of Sharing:** Mounted data can be shared between multiple containers at the same time. This means that one container can write to a mount while another container can read the same data.



Store Data on Host Machine

- Additionally, mounts are **not limited to the container host machine**. You can host **data mounts over the network**, for example by using the **NFS protocol**.
- **Volumes** are data mounts **managed by Podman**. **Bind mounts** are data mounts **managed by the user**.
- Both volumes and bind mounts can use the `--volume` or the equivalent `-v` parameter.
- **`--volume /path/on/host:/path/in/container:OPTIONS`**
- In the preceding syntax, the **`:OPTIONS`** part is optional. Note that you can specify host paths by **using absolute paths**, such as `/home/user/www`, **or relative paths**, such as `./www`, which refers to the `www` directory in the current working directory.
- Use `-v volume_name:/path/in/container` to refer to a volume.



Store Data on Host Machine

- Alternatively, you can use the `--mount` parameter with the following syntax:
`--mount type=TYPE,source=/path/on/host,destination=/path/in/container`
- The `--mount` parameter explicitly specifies the volume type, such as:
 - 1) `bind` for bind mounts.
 - 2) `volume` for volume mounts.
 - 3) `tmpfs` for creating memory-only, ephemeral mounts.
- The `--mount` parameter is the preferred way of mounting directories in a container. However, because the `-v` parameter is still widely used, this course uses both styles.
- Developers commonly use bind mounts for testing, or for mounting environment-specific files, such as property files. Because Podman manages the volume file system, use volumes to ensure consistent mount behavior across systems. Additionally, use volumes for advanced uses, such as mounting a remote volume over NFS.



1) Storing Data with Bind Mounts

- **Bind mounts** can **exist anywhere on the host system**.
- **For example**, to mount the **/www directory** on your **host machine** to the **/var/www/html** directory **inside the container** with the read-only option, use the following podman-run command:
- `podman run -d -p 8080:8080 --volume /www:/var/www/html:ro registry.access.redhat.com/ubi8/httpd-24:latest`
- Or you can use **--mount** instead of **-v**
- **For example**, `podman run -d --name devtest --mount type=bind,source="$(pwd)"/target,target=/app nginx:latest`



Troubleshoot Bind Mounts

- When you use bind mounts, you must configure **file permissions** and **SELinux access manually**.
- **SELinux** is an additional security mechanism used by Red Hat Enterprise Linux (RHEL) and other Linux distributions.

```
[student@workstation ~]$ podman run -d -p 8080:8080 --volume /www:/var/www/html
registry.access.redhat.com/ubi8/httpd-24:latest
382eec888f2ab1dc281d97d6c43c35b5bee2bf61925ac3e7f984b5b164563142
[student@workstation ~]$ podman exec -it thirsty_swanson /bin/bash
bash-4.4$ cd /var/www/html/
bash-4.4$ ls
ls: cannot open directory '.': Permission denied
bash-4.4$ exit
exit
[student@workstation ~]$ ls -ltr /www
total 0
```

- By default, the Httpd process has insufficient permissions to access the /var/www/html directory. This can be a file permission issue or an SELinux issue.
- To troubleshoot file permission issues, **use the podman unshare** command to execute the ls -l command.
- The **podman unshare** command **executes provided Linux commands** in a **new namespace** such as the one Podman creates for the container. This process maps user IDs as they are mapped in a new container, which is useful for troubleshooting user permissions

```
[student@workstation ~]$ ls -ld /www
drwxr-xr-x. 2 student student 6 Aug 14 17:19 /www
[student@workstation ~]$ podman unshare ls -ld /www
drwxr-xr-x. 2 root root 6 Aug 14 17:19 /www
```



Troubleshoot Bind Mounts

- To troubleshoot SELinux permission issues, inspect the /www directory SELinux configuration by running the ls command with the **-Z** option. Use the -d option to print only the directory information.
 - [user@host ~]\$ ls -Zd /www
 - system_u:object_r:**default_t**:s0:c228,c359 /www
- The output shows the SELinux context label system_u:object_r:default_t:s0:c228,c359, which has the **default_t** type.
- A container must have the **container_file_t SELinux type** to have access to the bind mount. SELinux is out of scope for this course.
- To fix the SELinux configuration, add the :z or :Z option to the bind mount:
 - Lower case z lets **different containers share access** to a bind mount.
 - Upper case Z provides the container with exclusive access to the bind mount.
- podman run -p 8080:8080 --volume /www:/var/www/html:Z registry.access.redhat.com/ubi8/httpd-24:latest

After adding the corresponding option, run the ls -Zd command and notice the right SELinux type.

```
[user@host ~]$ ls -Zd /www
system_u:object_r:container_file_t:s0:c240,c717 /www
```

Troubleshoot Bind Mounts



```
[root@workstation ~]# mkdir /www
[root@workstation ~]# ls -ldZ /www
drwxr-xr-x. 2 root root unconfined_u:object_r:default_t:s0 6 Aug 14 17:19 /www
[root@workstation ~]# chown student:student /www
[root@workstation ~]# ls -ldZ /www
drwxr-xr-x. 2 student student unconfined_u:object_r:default_t:s0 6 Aug 14 17:19 /www
[root@workstation ~]# exit
logout
[student@workstation ~]$ podman run -d -p 8080:8080 --volume /www:/var/www/html:Z registry.access.redhat.com/ubi8/httpd-24:latest
eaaaf11a0948f74029ecdcce498fbbca3d0d29b698151740342fb6ffcdabc4e31
[student@workstation ~]$ podman exec -it vigorous_kepler /bin/bash
bash-4.4$ cd /var/www/html/
bash-4.4$ exit
exit
[student@workstation ~]$ ls -lZd /www
drwxr-xr-x. 2 student student system_u:object_r:container_file_t:s0:c88,c161 6 Aug 14 17:19 /www
[student@workstation ~]$
```



2) Storing Data with Volumes (Named Volume)

- Volumes let Podman manage the data mounts. You can manage volumes by using the `podman-volume` command.
- To create a new volume use the following command: **`podman volume create <volume-name>`**
- To mount the volume into a container, refer to the volume by the volume name:
`podman run -p <host-port>:<container-port> --volume <volume-name>:<path-inside-container> <img-name>`
 - **`podman run -d -p 8080:8080 -v http-data:/var/www/html registry.access.redhat.com/ubi8/httpd-24:latest`**
- Another way by using `--mount`: **`docker run -d --name=<cont-name> -p <host-port>:<container-port> --mount source=<volume-name>,destination=<path-inside-container> <img-name>`**



Exporting and Importing Data with Volumes

- You can **import data from a tar archive** into an existing Podman volume by using the following command:
 - **podman volume import <volume-name><tar_archive>**
- You can also **export data from an existing Podman volume** and save it as a tar archive on the local machine by using the command:
 - **podman volume export <volume-name> --output <archive-name>**

3) Storing Data with a tmpfs Mount

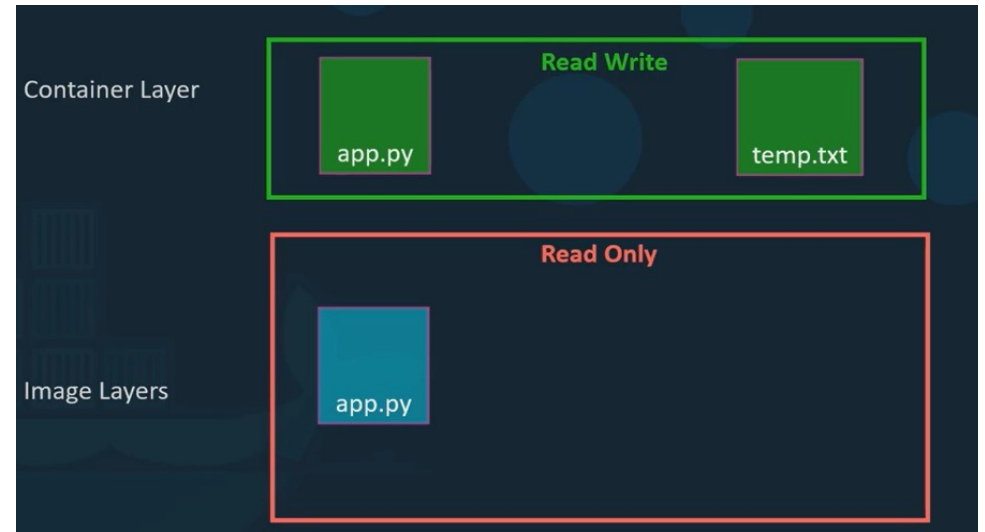
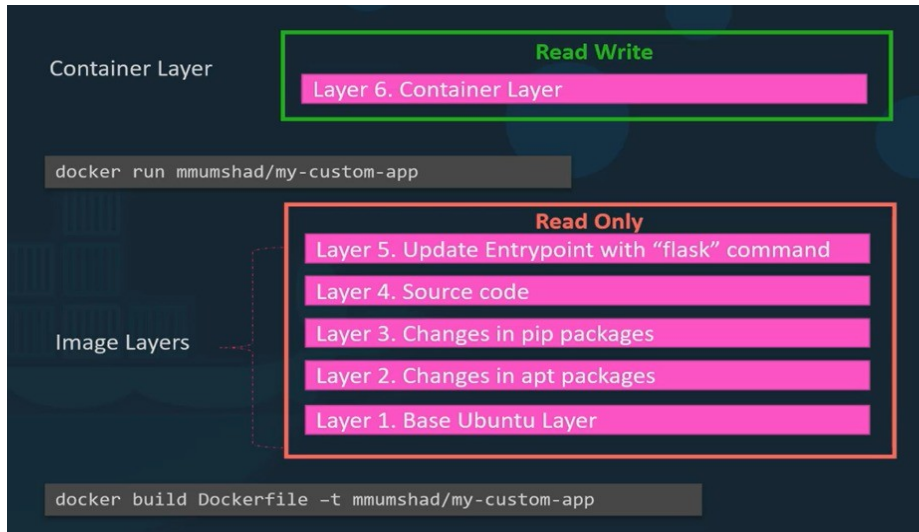
- Some applications cannot use the default COW file system in a specific directory for performance reasons, but the developers do not use persistence or data sharing for that directory.
- For such cases, you **can use the tmpfs mount type**, which means the data in a mount is ephemeral but **does not use the COW file system**:

```
podman run -e POSTGRESQL_ADMIN_PASSWORD=redhat --network lab-net \ --mount  
type=tmpfs,tmpfs-size=512M,destination=/var/lib/pgsql/data \  
registry.redhat.io/rhel9/postgresql-13:1
```



Dockerfile Layered Architecture

- When you run an image and generate a container, you add a **new writable layer**, also called the container layer, on top of the underlying layers.
- All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written to this writable container layer.





Two Types of External Data Storages

Volumes
(Managed by Docker)

Anonymous Volumes

Named Volumes

Docker sets up a folder / path on your host machine, exact location is unknown to you (= dev).
Managed via *docker volume* commands.

A defined path in the container is mapped to the created volume / mount.
e.g. /some-path on your hosting machine is mapped to /app/data

Great for data which should be persistent but which you don't need to edit directly.

Bind Mounts
(Managed by you)

You define a folder / path on your host machine.

Great for persistent, editable (by you) data (e.g. source code).



Anonymous Volumes	Named Volumes	Bind Mounts
Created specifically for a single container	Created in general – not tied to any specific container	Location on host file system, not tied to any specific container
Survives container shutdown / restart unless --rm is used	Survives container shutdown / restart – removal via Docker CLI	Survives container shutdown / restart – removal on host fs
Can not be shared across containers	Can be shared across containers	Can be shared across containers
Since it's anonymous, it can't be re-used (even on same image)	Can be re-used for same container (across restarts)	Can be re-used for same container (across restarts)



```
docker run -v /app/data ...
```

Anonymous Volume

```
docker run -v data:/app/data ...
```

Named Volume

```
docker run -v /path/to/code:/app/code ...
```

Bind Mount



volumes

```
docker volume create data_volume
```

Folder structure:

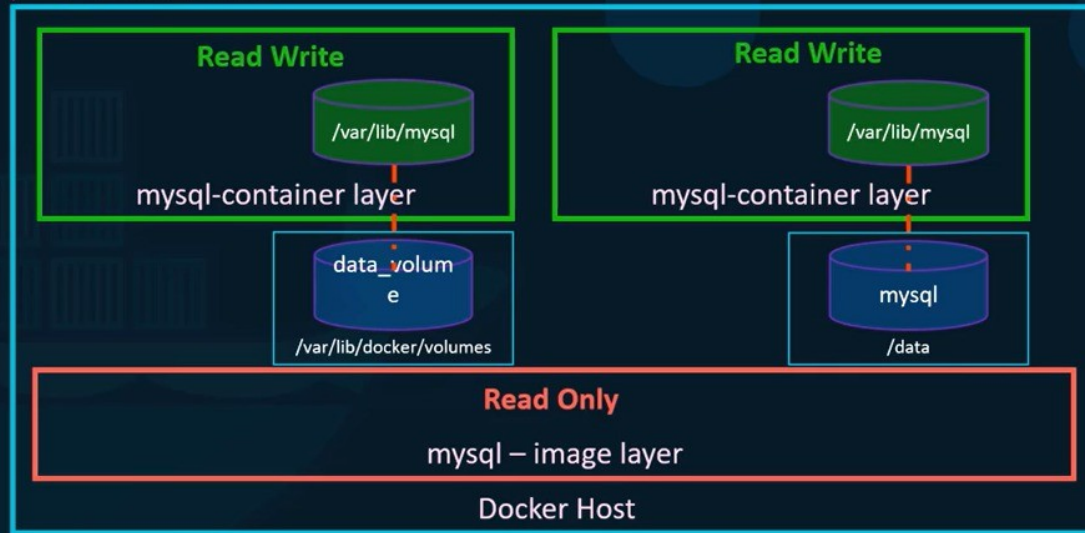
- /var/lib/docker
 - volumes
 - data_volume

```
docker run -v data_volume:/var/lib/mysql mysql
```

```
docker run -v data_volume2:/var/lib/mysql mysql
```

```
docker run -v /data/mysql:/var/lib/mysql mysql
```

```
docker run \
--mount type=bind,source=/data/mysql,target=/var/lib/mysql mysql
```



Section 5.3: Working with Databases





Stateful Database Containers

- **Database containers must maintain state** because they need to **persist data**. For this reason they are **called stateful containers**. This differs from other types of software, such as web APIs or proxies, that **do not need to maintain state** and therefore are **called stateless containers**.
- **It is more complicated for stateful containers to leverage the following benefits of containers:**
 - 1) **Portability:** Being able to move the container between different environments. Creating a container in the new environment is no longer enough and you must provide the container with up to date data. Portability affects applications when they are being developed.
 - 2) **Scalability:** Being able to increase performance by increasing the number of replicas of a container. Many database technologies are not designed to have data operated by more than one database process at a time.
 - 3) **Availability:** Being able to keep the system working in the event of one container crashing as long as that container has another replica running. Even though database systems have configurations for high availability, Podman alone does not provide the features needed for this type of configuration.
- Scalability and availability are not requirements for development or testing environments. They are addressed in production environments by using features offered by orchestration systems such as Red Hat OpenShift .



Good Practices for Database Containers

- **Use the VOLUME instruction in Containerfiles:** When you build a custom database container image it is a good practice to use the VOLUME instruction for the data directories used by the database. This is to avoid using the copy-on-write (COW) file system for performance reasons.
- When you create a container from an image that uses the VOLUME instruction, **Podman creates an anonymous volume** and attaches it to the container. This avoids using the COW file system, which is not performant for a database container.
- **Mount the data directory to a named volume:** Use a named volume to mount the data directory of your database to add data persistence across container recreations. Volumes are also more portable than bind mounts because the source directory does not need to exist in the host machine.
- **Create a database network:** If only containerized applications access your database then there is no need to expose your database to the host network. You can omit exposing the database port and use a Podman network to access the database.
- This provides better isolation for the database, because only applications that share the network have access to the database.

Lab: section 5.5





Videos

- <https://www.youtube.com/watch?v=zJ6WbK9zFpl>
- <https://www.youtube.com/watch?v=pg19Z8LL06w>